

MUFFLER: Secure Tor Traffic Obfuscation with Dynamic Connection Shuffling and Splitting

Minjae Seo*, Myoungsung You[†], Jaehan Kim[†], Taejune Park[‡], Seungwon Shin[†], and Jinwoo Kim[§]

*ETRI, Daejeon, Republic of Korea

[†]School of Electrical Engineering, KAIST, Daejeon, Republic of Korea

[‡]School of Artificial Intelligence, Chonnam National University, Gwangju, Republic of Korea

[§]School of Software, Kwangwoon University, Seoul, Republic of Korea

Abstract—Tor, a widely utilized privacy network, enables anonymous communication but is vulnerable to flow correlation attacks that deanonymize users by correlating traffic patterns from Tor’s ingress and egress segments. Various defenses have been developed to mitigate these attacks; however, they have two critical limitations: (i) significant network overhead during obfuscation and (ii) a lack of dynamic obfuscation for egress segments, exposing traffic patterns to adversaries. In response, we introduce MUFFLER, a novel connection-level traffic obfuscation system designed to secure Tor egress traffic. It dynamically maps real connections to a distinct set of virtual connections between the final Tor nodes and targeted services, either public or hidden. This approach creates egress traffic patterns fundamentally different from those at ingress segments without adding intentional padding bytes or timing delays. The mapping of real and virtual connections is adjusted in real-time based on ongoing network conditions, thwarting adversaries’ efforts to detect egress traffic patterns. Extensive evaluations show that MUFFLER mitigates powerful correlation attacks with a TPR of 1% at an FPR of 10^{-2} while imposing only a 2.17% bandwidth overhead. Moreover, it achieves up to 27x lower latency overhead than existing solutions and seamlessly integrates with the current Tor architecture.

I. INTRODUCTION

The Tor network, one of the most popular privacy networks [2], provides anonymity for internet users by routing traffic through a global network of volunteer-run relay nodes. Users establish a Tor circuit through three types of relays: entry, middle, and exit nodes. Traffic passes through this circuit to reach the intended services, with each node decrypting one layer of encryption to reveal the subsequent node in the path. This layered encryption ensures that no single node knows both the origin and final destination of the data in transit, providing anonymous and secure communication.

Users of the Tor network can access both *public* and *hidden* services anonymously without revealing their IP addresses. Public services are standard web servers accessible via both Internet and the Tor network. Conversely, hidden services [3], identified by their *.onion* addresses, are only

Minjae Seo and Myoungsung You contributed equally to this work. Jinwoo Kim is the corresponding author.

This work was partly supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by Korea government (MSIT) (No. 2021-0-00118, RS-2021-II210118: Development of decentralized consensus composition technology for large-scale nodes, 70%) and the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. 2022R1C1C1006967, 30%).

This paper is an extended version of our preliminary work [1].

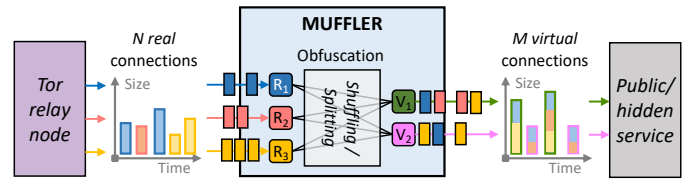


Fig. 1: MUFFLER dynamically obfuscates egress Tor traffic by mapping N real connections to M virtual connections.

accessible through the Tor network and hide the IP addresses of both users and servers from adversaries. To access hidden services, users first connect to designated rendezvous points to receive access points for these services. Subsequent communications between users and hidden services are routed through circuits connected to the specified access points, ensuring the end-to-end security and anonymity of both users and services.

Given Tor’s objective to facilitate anonymous communication for sensitive tasks, it has naturally become a target for various deanonymization attacks. Among these, the flow correlation attack [4] is particularly powerful, as it correlates unique characteristics of traffic flows, such as packet sizes and interval times, observed from the ingress and egress segments of a Tor connection. In recognition of the threat posed by flow correlation attacks, several obfuscation methods [5]–[12] have been developed to mitigate them. However, these methods are not well suited to the requirements of the Tor network due to the following two limitations:

Network Inefficiency. The majority of obfuscation methods currently employed to protect the Tor network focus on adding *crafted padding bytes* [6]–[10], [12]–[14] or *inter-packet delays* [6], [7], [13], [14] into packets, thereby hiding the original traffic patterns. However, they result in a significant increase in bandwidth consumption, which is a critical concern for the Tor network. Given that Tor relies on limited bandwidth resources provided by volunteers, any addition in bandwidth requirements is *not* trivial [15], [16]. Furthermore, intentional packet delays compromise communication latency and user experience, undermining the usability of these mechanisms.

Lack of Dynamic Obfuscation for Egress Segments. Effective obfuscation for the egress segment is crucial for maintaining anonymity as Tor encryption is fully decrypted when accessing *public* services. However, existing solutions solely

focus on obfuscation for the ingress segments, which fails to conceal critical traffic patterns. Recent machine learning-based attacks [17]–[19] have demonstrated high accuracy in deanonymizing users accessing public services through Tor networks. Similar vulnerabilities are present when accessing Tor *hidden* services. Although final Tor nodes, which are directly connected to hidden services, attempt to enhance anonymity by multiplexing client connections into a single one [3], the multiplexing schema remains static, employing a simple $N:1$ traffic pattern¹ regardless of network status (e.g., the number of connections). This static nature has enabled a recent machine learning-based attack to correlate users and their destination hidden services with over 99% accuracy [20].

Our Approach. In this paper, we introduce **MUFFLER**, a novel connection-level traffic obfuscation system designed to secure Tor traffic at the egress segment. As shown in Fig. 1, **MUFFLER** employs two obfuscation strategies: connection shuffling and connection splitting. These strategies map N real connections (e.g., TCP sessions) from the ingress segment into a set of M virtual connections established between the Tor final node and the target service, whether public or hidden. Packets from each real connection are relayed via a corresponding virtual connection, thereby implementing both $N:M$ shuffled and $1:M$ split traffic patterns within Tor egress segments. Additionally, **MUFFLER** dynamically adjusts the mapping between virtual and real connections in response to changing network conditions (e.g., the number of connections), hindering adversaries’ efforts to identify static mapping patterns. This dynamic connection-level obfuscation creates fundamentally different traffic patterns from those of ingress segments, thereby preventing existing flow correlation attacks and enhancing the anonymity of the Tor network.

Through the implementation of **MUFFLER**, we address the aforementioned limitations not tackled by existing solutions [6]–[10], [12]–[14]. It eliminates the need for padding bytes and inter-packet delays, effectively hiding original traffic patterns by dynamically relaying packets received from real connections through virtual connections, achieving an impressively low average bandwidth overhead of only 2.17%. Additionally, **MUFFLER** leverages recent kernel networking features [21] to avoid unnecessary kernel network stack processes during obfuscation, resulting in 15% and 24% improvements in mean and tail latency. Our connection-level obfuscation can seamlessly protect both public and onion services. Extensive evaluations show **MUFFLER**’s effectiveness in mitigating powerful flow correlation attacks targeting public and hidden services, achieving a True Positive Rate (TPR) of 1% at a False Positive Rate (FPR) of 10^{-2} .

Contributions. We make the following contributions:

- The design and implementation of **MUFFLER**, a defence system that obfuscates Tor traffic by dynamically mapping real connections into distinct virtual connections without the need for additional padding bytes or per-packet delays.

¹ N represents the number of original connections between users and the hidden service, while 1 denotes a single multiplexed connection between the guard node and the hidden service.

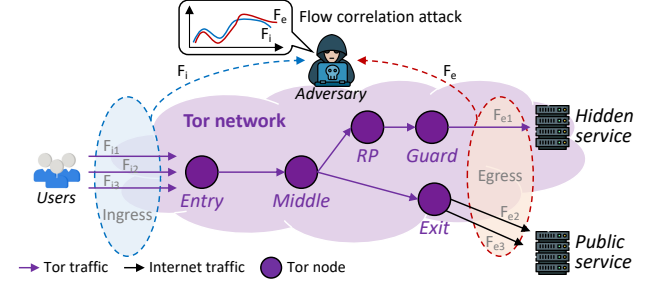


Fig. 2: Adversaries can correlate network flow pairs (i.e., ingress flows F_i and egress flows F_e) utilizing traffic features.

- The integration of a recent kernel networking feature that reduces kernel network stack processes typically required for obfuscation, thereby decreasing latency overheads.
- Extensive evaluations showing that **MUFFLER** prevents several flow correlation attacks, which are not adequately addressed by existing obfuscation methods, while imposing only minimal bandwidth and latency overheads.

II. BACKGROUND AND MOTIVATION

A. Flow Correlation Attacks

In most scenarios of flow correlation attacks, adversaries attempt to link traffic flows (e.g., TCP connections) observed from two points: *ingress flows* (F_i between users and entry nodes) and *egress flows* (F_e between guard/exit nodes and hidden/public services), as shown in Fig. 2. Those two points, with their pair of *associated flows* (F_i, F_e), carry the information of the real *identity* (i.e., IP address of the user or service), making them attractive targets for adversaries aiming to eavesdrop on Tor traffic. If adversaries collect enough traffic from both points, they can perform flow correlation attacks targeting public services or hidden services, uncovering the IP addresses of users through various traffic analysis techniques.

Flow Correlation Attacks on Public Services. Initially, several studies have used a statistical metric to measure the flow similarity of *associated flows* (F_i, F_e) observed from both *ingress* and *egress* points when accessing public services. For example, Sun et al. [22] used the Spearman correlation coefficient, a nonparametric measure that evaluates the statistical dependence between the rankings of two variables. They extracted *TCP header information*, particularly TCP sequence (SEQ) and acknowledgement (ACK) numbers, from each packet trace to conduct asymmetric correlation analysis, allowing adversaries to observe Tor traffic at both points (*ingress* and *egress*). However, they considered the case where there is a long-lasting Tor connection, requiring a long-term observation, rather than a short-lived Tor connection, which is more common in the real-world Tor network.

To address the limitations of the statistical metric, recent studies have adopted machine-learning techniques. For example, Nasr et al. [17] proposed a machine learning-based flow correlation attack, DeepCorr, which can correlate *associated flows* (F_i, F_e) with high accuracy using short-term observations of Tor connections. Due to the encryption for Tor packets,

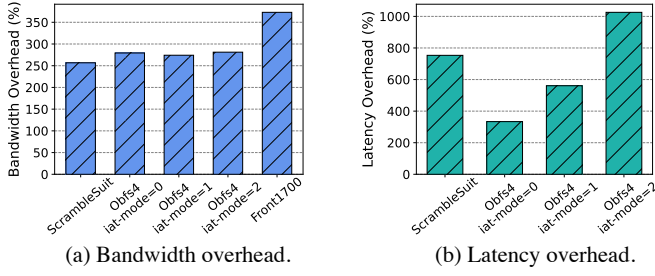


Fig. 3: Overheads of existing client-side obfuscation methods.

they leveraged deep neural networks (DNNs) to learn hidden patterns within Tor traffic. They found dominant characteristics in *packet sizes* and *inter-packet delays* when correlating Tor traffic. Their evaluations demonstrated that DeepCorr can correlate associated Tor flows with 96% accuracy.

Flow Correlation Attacks on Hidden Services. Recently, a flow correlation attack targeting hidden services has been developed. This attack, known as the SUMo attack [20], leverages a novel technique called the Sliding Subset Sum algorithm to measure flow similarity. The SUMo attack utilizes the absolute packet arrival times to correlate flows, modeling packets received by clients and transmitted by hidden services as bounded time series. This approach allows adversaries to calculate the similarity score of flows by aligning *packet sizes* within sliding windows, enhancing its efficacy in identifying patterns. Although multiple client connections from Tor relay nodes between clients and a hidden service are multiplexed in a single connection, it still exposes the traffic feature of packet sizes in a bounded time series.

Attack Vector. Flow correlation attacks on Tor, leveraging traffic features such as *TCP header context*, *packet sizes*, and *inter-packet delays*, present a significant privacy concern. This situation highlights an imperative demand for robust defense mechanisms capable of obscuring these attack vectors.

B. Motivation

To prevent adversaries from exploiting the aforementioned attack vectors, various traffic obfuscation solutions [6]–[10], [12], [13] have been developed. These solutions focus on client-side obfuscation, which masks Tor ingress traffic between users (clients) and Tor entry nodes. For example, Tor’s official client-side obfuscation methods, such as ScrambleSuit [14] and obfs4 [13], utilize *padding* and *timing delays* to obscure identifiable features. ScrambleSuit sends MTU-sized packets, adding padding when data is less than the MTU to randomize packet sizes. Obfs4 enhances this approach with stronger cryptographic protections during the initial TLS handshake between users and entry nodes. It employs the “iat-mode” (Inter-Arrival Time mode) to adjust traffic timing, offering settings to disable (0) or enable (1 or 2) intentional delays, thus improving obfuscation effectiveness. FRONT [10] is an advanced client-side obfuscation method that hides the “front” part of web traffic with random dummy packets.

Network Inefficiency. Despite the enhancements in user protection brought by adopting client-side obfuscation methods,

concerns remain about network inefficiencies. To formalize this, we define the bandwidth and latency overhead, partially borrowed from [10] as follows: We define a sequence of n packets denoted by $P = \{p_1, p_2, \dots, p_n\}$, where p is a packet. Each packet p has (t_i, L_i) , where t_i is the timestamp and L_i is the length of the i -th packet, respectively. We also define $|P| = \sum_{i=1}^n L_i$ by the total length of a sequence of packets. Here, let P denote the original sequence and P' the obfuscated sequence after applying a certain obfuscation solution D . The bandwidth and latency overhead are defined as follows: The *bandwidth overhead* $O(D)$ of defense D on P is the total padding length (i.e., increased packet length) divided by the total packet length of the original sequence: $O(D) = \frac{|P'| - |P|}{|P|}$. The *latency overhead* $T(D)$ of defense D on P is the extra time taken to transmit real packets, divided by the original transmission time. Denoting the timestamp of the last real packet in P' as t_k , then we have: $T(D) = \frac{t_k - t_n}{t_n}$.

With this definition, we performed a 30-minute web browsing experiment to measure the bandwidth and latency overhead of ScrambleSuit, obfs4 with various iat-mode settings, and FRONT. Fig. 3 (a) shows that these techniques result in approximately a 250% bandwidth overhead. Additionally, Front1700, which uses a ratio of 1,700 dummy packets for every 10,000 real packets, results in the highest bandwidth overhead, exceeding 350%. Given the limited bandwidth of the Tor network, any increase in bandwidth consumption is significant. Furthermore, Fig. 3 (b)² indicates that ScrambleSuit alone causes a latency overhead of over 700%. The iat-mode settings further increase latency, with overheads of approximately 300%, 600%, and 1,000% for iat-mode=0, 1, and 2, respectively. These results highlight that existing client-side obfuscation methods introduce substantial bandwidth and latency overhead, exacerbating network performance issues.

Lack of Dynamic Obfuscation for Egress Segments. Existing solutions focus on obfuscating traffic at Tor ingress segments (between users and entry nodes). However, this client-side obfuscation alone is insufficient for protection against flow correlation attacks, as it fails to hide critical correlation features at Tor egress segments. This is particularly concerning as all protections provided by Tor networks are stripped at this point, revealing the IP addresses of public services that users connected to. This limitation enables machine-learning-based attacks [17]–[19] to correlate users with public services with high accuracy. Moreover, hidden services, traditionally considered secure, face similar threats. As shown in Fig. 2. Tor guard nodes multiplex multiple user connections into a single one using a fixed $N:1$ mapping. This schema does not dynamically adjust to changes in network conditions, such as the number of connections, making it ineffective against evolving threats. Such static obfuscation methods allow advanced techniques like SUMo [20] to identify correlation features within short time slots, achieving a 99% accuracy rate in linking users to their destination hidden services.

²Note that FRONT is excluded from the latency evaluation as it obfuscates collected traffic traces offline rather than real-time traffic.

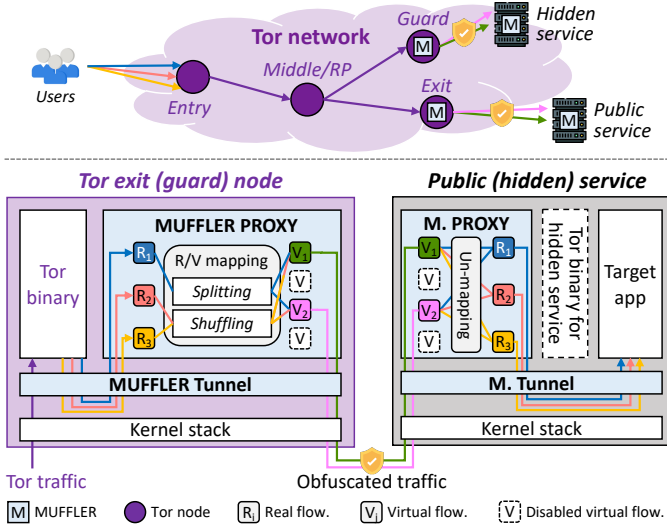


Fig. 4: MUFFLER is deployed between the final Tor relay node and the target service (hidden or public).

III. MUFFLER DESIGN

A. Design Considerations

To address the aforementioned limitations, we structure MUFFLER around the following design considerations (DC):

DC1. Efficient Obfuscation without Intentional Overheads. Existing solutions rely on intentional data padding or timing delays. These methods result in significant bandwidth consumption and increased latency. Thus, the proposed system should obfuscate Tor traffic without intentional overheads while providing the same level of security guarantee.

DC2. Robust and Dynamic Egress Traffic Obfuscation. Existing solutions lack robust server-side obfuscation for the Tor network, particularly for Tor hidden services and the traffic between exit nodes and public services. This limitation allows adversaries to identify users accessing both hidden and public services. Consequently, the proposed system should implement dynamic server-side obfuscation that comprehensively secures Tor egress traffic directed to both hidden and public services, adapting in real-time to the prevailing network conditions.

DC3. Efficient and Seamless Integration with Tor. The compatibility of obfuscation systems with the existing Tor ecosystem is essential for their broad deployment. Therefore, the proposed system should be designed for seamless integration with the Tor ecosystem without requiring modifications to the Tor protocol or the Tor binary. Also, it should obfuscate and process egress traffic in an optimized way to avoid becoming a bottleneck on the Tor network.

B. Threat Model and Assumptions

Threat Model. The goal of adversaries is to deanonymize users by correlating a pair of *associated flows* (F_i , F_e), including *ingress flows* (F_i) and *egress flows* (F_e), as shown in Fig. 2. They compare several unique characteristics of Tor traffic (e.g., packet sizes and timings) instead of attempting to decrypt the content of Tor traffic. To achieve their goal, we

consider adversaries who have capabilities to eavesdrop on Tor traffic (i.e., ingress and egress flows) passively.

We believe this scenario is practical in the real world. In recent years, many threat actors are running hundreds of malicious Tor nodes in order to intercept Tor traffic [23]–[25]. Adversaries can further leverage BGP hijacking attacks, which becomes increasingly frequent, by a means to redirect the Tor traffic to themselves [22]. There might also be more powerful adversaries, such as governmental agencies, who can perform wiretapping attacks on multiple Internet ASes/IXPs [22], [26]–[29] or intercontinental fiber optics [30].

Deployment Scenario. We assume a scenario where users seek anonymous access to public or hidden (onion) services via Tor networks to circumvent censorship and Internet tracking. To protect these users from flow correlation attacks, we propose that MUFFLER be deployed on machines operating target services—public or hidden—and on the *egress Tor nodes* directly connected to these services. For hidden services, the egress nodes are defined as Tor guard nodes, whereas for public services, they are Tor exit nodes. This deployment model is practical, as operators of hidden services and users can designate a set of predefined guard nodes through their Tor configuration files (`torrc`) [31]. Furthermore, considering the pivotal role of exit nodes in forwarding Tor traffic to the Internet, several reputable organizations and privacy-conscious entities already manage trusted exit nodes [32], [33] equipped with enhanced security features. MUFFLER can be effectively installed on these exit nodes to offer additional protection for users accessing security-enhanced public services.

C. MUFFLER Overview

Fig. 4 shows the overall architecture of MUFFLER comprising two main components: the MUFFLER Tunnel and the MUFFLER PROXY. These components are adeptly deployed on target servers (either hidden or public servers) and an egress Tor relay node directly connected to these servers. This deployment strategy ensures the transparent and efficient obfuscation of egress traffic. The operation of MUFFLER is structured into two main phases:

Obfuscation Phase. This phase begins with the Tor binary on the Tor egress node processing incoming Tor traffic from its previous relay node. To achieve transparent obfuscation, the MUFFLER PROXY runs as a reverse proxy, intercepting traffic from the Tor binary and sending obfuscated traffic to target services. When doing so, this proxy handles non-onion traffic for securing connections between exit nodes and public services, as well as onion traffic for protecting connections between guard nodes and hidden services. While the Tor binary and the MUFFLER PROXY run on the same host, the traditional local-host communication can lead to multiple packet copies and repetitive kernel network stack operations. Thus, the MUFFLER Tunnel facilitates socket-level packet redirection. Data transmitted from the Tor binary's socket is directly inserted into the corresponding socket of the MUFFLER PROXY. This redirection operates before packetization, avoiding extra packet copies and kernel stack operations.

As shown in Fig. 4, the MUFLER employs connection-level obfuscation by mapping real TCP connections—those between users and services—to a set of virtual connections established in conjunction with its paired proxy on the server side. This approach allows packets from real connections to be transmitted through corresponding virtual connections. Here, MUFLER utilizes two novel mapping strategies: (i) connection shuffling and (ii) connection splitting. In connection shuffling mode, packets from N real connections are shuffled across M virtual connections, thereby creating merged traffic patterns. In contrast, connection splitting mode spreads packets from a single real connection over M multiple virtual connections, creating evenly distributed traffic patterns. Moreover, the mapping between real and virtual connections dynamically adjusts during runtime based on each connection's state, further hindering adversaries from discovering correlation patterns.

De-obfuscation Phase. On the server side, the obfuscated packets within these virtual connections are restored to their original real connections. The MUFLER PROXIES on both the egress node and the target server synchronize their connection mapping states through our control protocol. Based on these mapping states, shuffled packets are de-obfuscated to their corresponding real connections. Similarly, packets distributed across various virtual connections are consolidated into a single real connection. These de-obfuscation processes maintain data integrity for server processing. The MUFLER PROXY subsequently forwards de-obfuscated packets to the intended service application (e.g., web services). For hidden services, de-obfuscated packets are relayed back to the Tor binary for final onion processing before reaching the application. Throughout these transmission steps, the MUFLER Tunnel ensures direct data transfer from source to destination sockets, effectively bypassing unnecessary kernel stack involvements.

By implementing the above processes, MUFLER addresses the three design considerations. Specifically, it maps real connections to a set of virtual connections, which exhibit distinct patterns, such as variations in the number of connections and SEQ/ACK numbers. This connection-level obfuscation effectively conceals the patterns of real connections without relying on padding bytes or timing delays (DC1). Furthermore, MUFLER runs between the Tor egress nodes and their target services, protecting Tor egress traffic destined for both public and hidden services, thus mitigating the risk of correlation attacks in the egress segments (DC2). Additionally, our proxy-based design enables seamless integration with the existing Tor ecosystem without any modifications. The MUFLER Tunnel complements this design by avoiding the overheads associated with traditional proxy-based obfuscations (DC3).

D. MUFLER Tunnel

MUFLER adopts a proxy-based approach that intercepts outgoing packets from the Tor binary. Although this design aligns transparently with the Tor binary, it introduces several inefficiencies during packet redirection to the MUFLER PROXY. Specifically, this redirection incurs two packet copies and two kernel stack processes, which consume host system

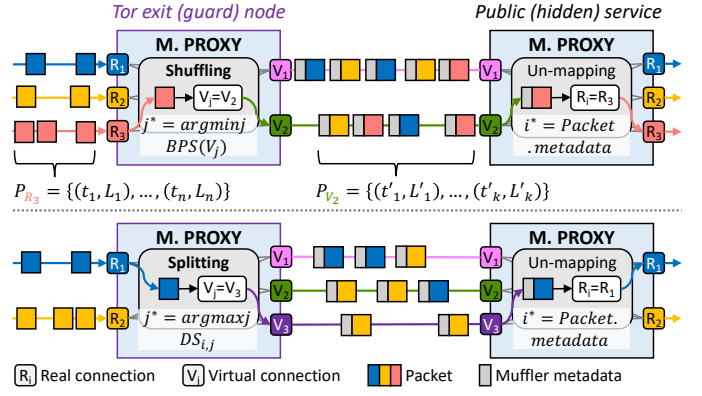


Fig. 5: An overview of connection shuffling (upper) and connection splitting (lower) of the MUFLER PROXIES.

resources and contribute to increased latency [34], [35]. To address these issues, MUFLER employs the MUFLER Tunnel that enables traffic redirection at the socket level between the Tor binary and the MUFLER PROXY, thereby avoiding these overheads, as shown in Fig. 4.

The MUFLER Tunnel runs on the kernel space and monitors socket system calls (e.g., *connect* and *send*) invoked by the Tor binary. When the Tor binary attempts to create a socket with the target service, this tunnel intercepts the relevant system call. It then records the original destination of the socket and modifies the system call's parameter, making the Tor binary to establish a socket with the MUFLER PROXY instead of the target service. When the Tor binary later attempts to send a new data through the created socket, this tunnel intercepts the data and redirects it from the source socket's Tx queue to the destination socket's Rx queue, thus avoiding the traditional kernel stack operations and unnecessary packet copies. After obfuscation, the MUFLER PROXY utilizes the recorded destination information to send the obfuscated traffic to the original destination through corresponding virtual connections, ensuring proper routing to the target services.

E. Obfuscation with Connection Shuffling and Splitting

To ensure robust security for Tor egress traffic, MUFLER maps real connections to virtual connections through connection-level obfuscation. As shown in Fig. 5, it selects a proper mapping strategy among the following two strategies based on the current number of real connections (N).

Connection Shuffling. When N exceeds a shuffle threshold (S)—a sufficient volume for effective shuffling—the MUFLER PROXY runs in connection shuffling mode. This mode adjusts the number of active virtual connections (M) to meet the following schema: $M = \min(\lfloor \alpha N \rfloor, M_{\min})$ where α is a shuffling factor less than one, ensuring a lower number of virtual connections than real connections, and $M_{\min}$ is the minimum number of virtual connections maintained for robust obfuscation. As shown in the upper part of Fig. 5, packets from N real connections are relayed through M virtual connections, creating an $N : M$ shuffled traffic pattern where each virtual connection receives a mix of packets from multiple real

connections. Also, the selection of a j -th virtual connection for each packet considers the current bytes per second (BPS) of each virtual connection using the following schema: $j^* = \arg \min_j BPS(V_j)$ where $BPS(V_j)$ indicates the BPS of a virtual connection. This schema ensures that incoming packets are distributed evenly across virtual connections, preventing any single virtual connection from becoming a bandwidth bottleneck and revealing a unique pattern.

Connection Splitting. Although connection shuffling effectively hides the correlation features of real connections, it becomes less effective with a smaller number of real connections and is infeasible with only one real connection. To address this limitation, the MUFFLER PROXY switches to connection splitting mode when N falls below the shuffling threshold (S). In this mode, it activates new virtual connections according to the following schema: $M = \max(\beta N, M_{\min})$ where β is a splitting factor greater than one, designed to increase the spread of traffic across a greater number of virtual connections. As shown in the lower part of Fig. 5, this mode splits packets from a real connection evenly across the M virtual connections, creating $1 : M$ split traffic patterns. In addition, the MUFFLER PROXY leverages a dissimilarity score to optimize traffic distribution: $DS_{i,j} = |BPS(R_i) - BPS(V_j)|$ where $BPS(R_i)$ and $BPS(V_j)$ represent the BPS of i -th real and j -th virtual connections, respectively. The selected virtual connection for packet relay maximizes this dissimilarity score: $j^* = \arg \max_j DS_{i,j}$, ensuring that the traffic distribution across virtual connections does not reflect the bandwidth patterns of the real connection. This connection splitting mode hides individual correlation patterns from adversaries even when real connections are limited.

Our connection-level obfuscation generates correlation features that are fundamentally distinct from those observed at the Tor ingress segments, including differences in TCP context, packet sizes, and inter-packet delays. As shown in the upper part of Fig. 5, individual packets within the packet sequences of three real connections ($P_{R_1}, P_{R_2}, P_{R_3}$) are multiplexed into two virtual connections (P_{V_1}, P_{V_2}), resulting in obfuscated packet sequences. Assume that adversaries observe a specific correlation feature such as BPS for the obfuscated sequence P_{V_2} . Consequently, $BPS(P_{V_2})$ is calculated as $|P_{V_2}| / (t'_k - t'_1)$, where k denotes the last packet index of the obfuscated sequence P_{V_2} . It becomes clear that $BPS(P_{V_2})$ is not equivalent to $BPS(P_{R_1})$, $BPS(P_{R_2})$, or $BPS(P_{R_3})$, thereby rendering adversaries' flow correlation attacks ineffective.

Connection Un-mapping. To maintain the integrity of service processing, the MUFFLER PROXY at the target service should correctly dispatch packets from virtual connections back to their corresponding original real connections (i.e., un-mapping). The MUFFLER PROXIES on both ends facilitate this process by exchanging control commands through their virtual connection. In the current design, a control command is 8-byte long, including four 2-byte fields: a command type field, two operand fields, and one reserved field. When relaying a data packet from a real connection, a *relay* type command is utilized. As shown in Fig. 5, this command is embedded within

each packet's payload as metadata, including real connection IDs. The receiving proxy utilizes this information to accurately dispatch packet data from each virtual connection to the appropriate sockets linked with their original real connections. Additionally, control commands are used for synchronizing configurations between MUFFLER PROXIES. For instance, the creation of a new virtual connection triggers the exchange of a *create* type command between proxies prior to executing TCP handshakes. A *keep-alive* type command is also used to maintain deactivated virtual connections. Note that since *relay* commands are embedded into the actual data packets, they may introduce some bandwidth overhead. However, with each *relay* command adding only 8 bytes, this overhead is considered negligible, when compared to previous data padding methods [10], [13] that add between 500 to 1,000 dummy bytes per packet to create fixed-length traffic patterns.

IV. EVALUATION

In this section, we first evaluate the bandwidth and latency overhead associated with MUFFLER compared to existing solutions. Next, we evaluate the effectiveness of MUFFLER in obfuscating Tor traffic against several flow correlation attacks.

A. Prototype Implementation

We have developed a full prototype of the MUFFLER PROXY, leveraging the core functionality of HAProxy [36] and extending it using the Go language. The MUFFLER PROXY consists of client and server components. These components initiate multiple long-lived TLS (or TCP) connections, referred to as base connections, which are utilized to create virtual connections. To facilitate the division of a single base connection into multiple virtual connections, we implemented a set of control commands: *create*, *remove*, *relay*, and *keep-alive*, as described in Section III-E. Additionally, the MUFFLER Tunnel leverages three types of eBPF programs [37], [38]. These programs are attached to the Tor binary and the MUFFLER PROXY to monitor socket system calls, modify system call arguments, store socket descriptors, and redirect data from source sockets to destination sockets.

B. Experimental Environment

Private Tor Testbed. To evaluate the feasibility of MUFFLER while mitigating potential impacts on real-world Tor users, we create a private Tor testbed atop a Kubernetes cluster, utilizing three physical machines. The testbed includes a number of Tor relay nodes—entry, middle, exit nodes, and directory authorities. Each machine is equipped with two Intel Xeon Silver 4114 CPUs and 64GB of memory. This configuration ensures an isolated environment distinct from the real Tor network, facilitating a controlled comparison of performance between MUFFLER and existing studies. Moreover, to further isolate resource usage and minimize interference, we run public and hidden services on separate servers equipped with an Intel i9-10900X CPU with 256GB of memory.

Defense Settings. To evaluate the effectiveness of MUFFLER within our testbed, we configure its parameters as follows. We

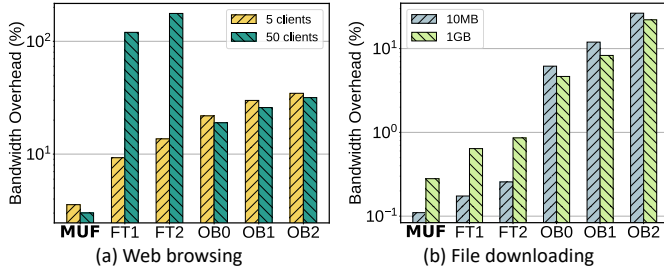


Fig. 6: Bandwidth overhead of obfuscation solutions. *FT1* and *FT2* indicate FRONT 1700 and FRONT 2500. *OB0-2* represents obfs4 with iat-mode of 0, 1, and 2, respectively.

set the shuffling threshold (S) to four, meaning that **MUFFLER** runs on the connection shuffling mode when N exceeds four. Additionally, we configure the settings of α , β , and $M_{\min}$ (see Section III-E) to 0.1, 2, and 3, respectively, to optimize performance and obfuscation effectiveness. As **MUFFLER** is the first egress-side obfuscation for Tor, we compare it with the following two popular ingress-side obfuscation systems. We first consider obfs4 across different inter-arrival time modes (iat-mode 0, 1, and 2), which manage the inter-packet delays. FRONT [10] is a state-of-the-art system that obfuscates the initial (front) part of web traffic traces using dummy padding data. We consider two variants of FRONT—FRONT 1700 and FRONT 2500—where the numbers represent the number of dummy packets per 10,000 real packets, as specified in the original paper [10]. As the specific size of the dummy packets used in FRONT is not directly stated in the paper, we set the size of dummy data to MTU size (1,500 bytes), which is a prevalent setting in existing studies [6], [39].

C. Bandwidth Overhead

We evaluate bandwidth overhead using the same definitions presented in Section II-B. We run two types of applications commonly used on Tor networks: website browsing and file downloading, as shown in Fig. 6.

Website Browsing. In this experiment, we analyze website browsing performance using two widely used microservice applications [40], [41], simulating an environment with 5 clients and 50 clients to mimic realistic scenarios. Fig. 6 (a) shows the evaluation results. **MUFFLER** employs the connection shuffling mode across virtual connections, which introduces only minimal overheads of 3.56% and 3.01% under 5 and 50 clients, respectively. This overhead consists of our *relay* commands embedded into the payload of each packet. In contrast, existing solutions show significant overheads due to their padding-based obfuscation strategies. FRONT in its 1700 and 2500 settings shows significant bandwidth overheads of 9% and 13% under 5 clients. This overhead further increases to exceeding 100% when handling 50 clients. In addition, obfs4 under different iat-modes shows a stepwise escalation in bandwidth usage for both 5 clients and 50 clients. Specifically, with 5 clients, the bandwidth usage increases to 21.84%, 29.93%, and 34.61%, respectively, for each iat mode.

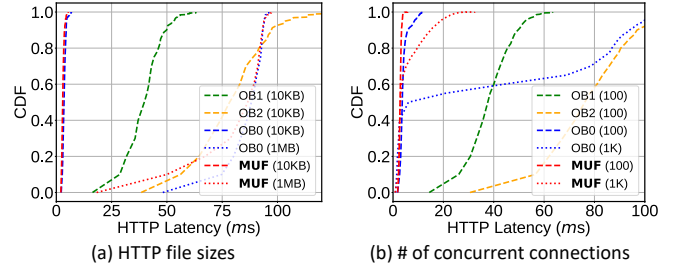


Fig. 7: The web browsing latency measurements of **MUFFLER** with the comparison of existing obfuscation methods.

File Downloading. In scenarios involving file downloads, characterized by long-lasting HTTP connections and large-sized packets, **MUFFLER** demonstrates exceptionally low bandwidth overheads, below 1%, as shown in Fig. 6 (b). This minimal overhead stems from the fact that the size of our relay commands is significantly smaller than the overall data packet size during such downloads. FRONT exhibits slightly higher overheads than **MUFFLER**, under 2%, because it only adds padding data to the initial part of the HTTP connection. On the other hand, obfs4 experiences significantly higher bandwidth overheads across all file sizes due to its method of inserting substantial amounts of dummy data to obscure traffic patterns. Despite handling large-sized packets, obfs4’s approach of extensive padding results in consistent and considerable overheads—averaging 5.41% for iat-mode 0, 10.13% for iat-mode 1, and 24.27% for iat-mode 2.

D. Latency Overhead

Here, we evaluate the latency overhead introduced by **MUFFLER**. Using the same testing environment as in previous evaluations, we measure HTTP round trip latency from the moment the Tor binary within the exit node sends an HTTP request to when it receives the HTTP response from the public web service. For a fair comparison, obfs4 is configured to operate between the Tor exit node and the target service, mirroring **MUFFLER**’s setup. Note that FRONT is excluded from this evaluation as it obfuscates already collected traffic traces offline, rather than real-time traffic during run-time.

Fig. 7 (a) shows the evaluation results for varying HTTP response sizes. When handling 10KB HTTP traffic, **MUFFLER** achieves a mean latency of 2.887 *ms*, outperforming obfs4-iat2 by 27x. In addition, compared to obfs4-iat0, which incurs minimal delay at the cost of degraded security, **MUFFLER** demonstrates a 17.1% reduction in latency. These performance improvements become more evident with larger HTTP traffic sizes. When handling 1MB of traffic, **MUFFLER** significantly outperforms all other solutions by a significant margin. Fig. 7 (b) shows the evaluation results for different levels of concurrent connections with 10KB HTTP files. While previous solutions struggle under high connection loads due to their reliance on inefficient obfuscation methods, **MUFFLER** outperforms other solutions by up to 27x, achieving a mean latency of 3.4*ms* and a P90 latency of 14.9*ms* under 1K connections.

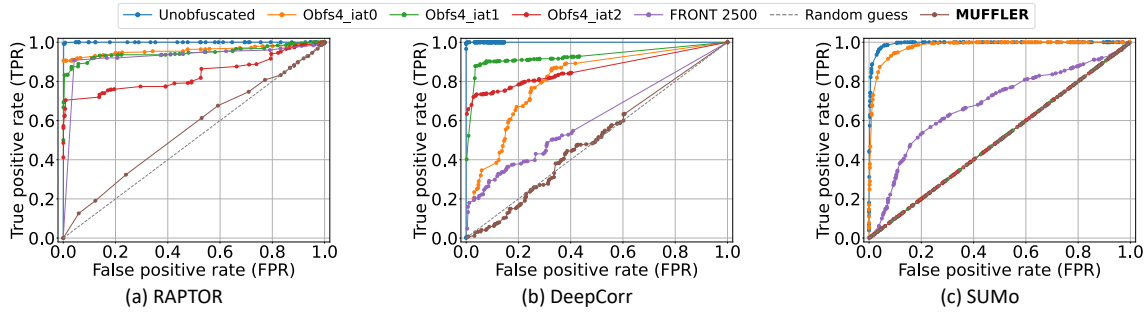


Fig. 8: The ROC curves of the performance of the flow correlation attacks across obfuscation methods.

E. Obfuscation Effectiveness

We evaluate the security effectiveness of MUFLER against several powerful flow correlation attacks within our private Tor network, comparing it to previous obfuscation methods across two scenarios: accessing public services and hidden services. In the first scenario, we employ RAPTOR [22]—a statistical metric-based attack, and DeepCorr [17]—a machine learning-based attack. We collect 500 ingress and egress flows during web browsing to train the DeepCorr model and another set of 500 flows for testing RAPTOR and DeepCorr. During traffic collection, we use various numbers of concurrent flows from 2 to 100. Note that the training for DeepCorr is performed exclusively on non-obfuscated flows to establish a baseline, while the evaluation phase for both attacks uses obfuscated flows³. The hyperparameters for these models are selected according to the original studies to ensure optimal attack performance. The second scenario focuses on hidden services, where we employ SUMo [20]—an advanced flow correlation attack specifically designed to identify users accessing Tor hidden services. We deploy multiple hidden services to our private Tor network by leveraging popular microservice web applications [40], [41], simulating realistic hidden web service traffic. We collect 500 ingress/egress flows when accessing these hidden services with various concurrent connections ranging from 2 to 100 to generate test traffic.

To quantify the effectiveness of the flow correlation attacks, we measure the True Positive Rate (TPR) and False Positive Rate (FPR) of the adversaries' predictions. A *true positive* is identified when the adversaries correctly correlate an actual correlated flow pair, while a *false positive* is identified when two irrelevant flows are mistakenly predicted as correlated. In evaluating MUFLER, each flow at the egress segment (virtual connection) contains multiplexed packets, making it unsuitable to directly match the egress flow with its correlated ingress flow (real connection). For a fair comparison of MUFLER with previous solutions, we define the ground truth for a correlated flow pair (ingress and egress) based on the *averaged flow similarity*. Specifically, the similarity of a flow pair is calculated by the Euclidean distance in packet size and inter-

packet delay, averaged within a specific time window. We consider an ingress flow (real connection) and an egress flow (virtual connection) are correlated if their pair exhibits the highest averaged flow similarity.

Public Services. Fig. 8 (a) shows the effectiveness of various obfuscation methods against sophisticated flow correlation attacks. In the absence of obfuscation, RAPTOR and DeepCorr exhibit high TPRs of 99% and 100%, respectively, at an FPR of 10^{-2} . MUFLER, however, demonstrates exceptional defense capabilities; against RAPTOR, depicted in Fig. 8 (a), it achieves a TPR of 0%, effectively mitigating this statistical metric-based attack. This result underscores the robustness of MUFLER in concealing critical TCP sequence and acknowledgment numbers by rerouting packets through distinct virtual connection contexts. Against the DeepCorr attack, which leverages machine learning techniques, MUFLER outperforms baseline methods, including FRONT2500, obfs4-iat0, obfs4-iat1, and obfs4-iat2, each of which achieves TPRs of 9%, 26%, 77%, and 64%, respectively. MUFLER maintains a TPR of only 1% at the same FPR level. This superior performance is attributed to MUFLER's dynamic adjustment of mapping between real and virtual connections, effectively shuffling and splitting traffic features like BPS, PPS, and inter-packet delays across various virtual connections. Such dynamic connection-level obfuscation hinders DeepCorr's ability to match flow-level features, thereby defending against both statistical and machine learning-based flow correlation attacks.

Hidden Services. In the context of hidden services, SUMo, designed to correlate traffic flows to hidden services, achieves a TPR of 88% without obfuscation, as shown in Fig. 8 (c). While obfs4-iat0 allows a significant correlation with a TPR of 64.8%, it performs markedly better under other configurations like obfs4-iat1 and obfs4-iat2, where SUMo is less effective. This ineffectiveness arises because SUMo is specifically designed to de-multiplex merged traffic flows through sliding subset sums—a technique that struggles against the randomly padded and delayed traffic patterns introduced by methods like obfs4 and FRONT [20]. Note that MUFLER mainly runs in connection splitting mode during this evaluation as the Tor guard node merges multiple flows into single one. While the number of real connections is limited, MUFLER severely diminishes SUMo's effectiveness by splitting packets from each real connection across multiple virtual connections, considering the individual BPS patterns of each connection.

³We conduct an additional experiment training and evaluating the DeepCorr model on obfuscated flows, but it shows only a negligible F1-score increase of 0.004 due to MUFLER's dynamic shuffling and splitting. Thus, we omit this evaluation, focusing instead on a more realistic adversarial setting.

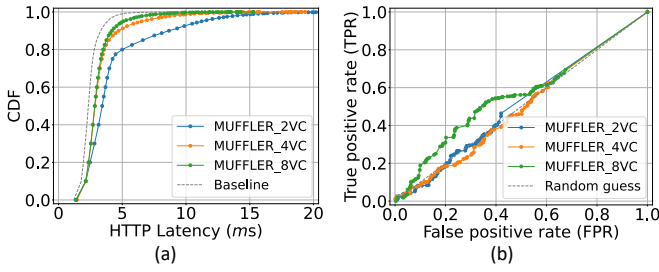


Fig. 9: The latency and obfuscation effectiveness under different shuffling factors. The baseline means direct 1:1 mapping.

This leads to a drastic reduction in SUMo’s TPR to 1% at an FPR of 10^{-2} , rendering its accuracy to levels comparable to random guessing. MUFLER’s effectiveness stems from its dynamic connection splitting mechanism that spreads packets to different virtual connection sets and adjusts the mapping in run-time, preventing SUMo from identifying consistent features for sliding subset sum calculations.

F. Impact of Shuffling Factor

We further assess the impact of varying the shuffling factor (α) on MUFLER’s performance against flow correlation attacks and the associated overhead. For this, we employ 100 concurrent real connections and map them to 2, 4, and 8 virtual connections using different shuffling factors (i.e., $\alpha = 0.02, 0.04$, and 0.08). We focus on measuring HTTP latency and the security effectiveness against the advanced attack, DeepCorr.

As shown in Fig. 9 (a), the latency overhead increases linearly with the degree of shuffling. Specifically, in 2 virtual connections setting, which offers the most robust security, MUFLER shows a mean latency of $3.69ms$ and P90 latency of $8.66ms$. These values represent 49.1% and 161% increases in overhead, respectively, compared to the baseline that uses one-to-one mapping without security. This overhead is caused by virtual connections becoming bottlenecks under high degrees of shuffling, resulting in packet drops. In the 8 virtual connections setting, as shown in Fig. 9 (b), there is a slight reduction in security effectiveness, achieving a TPR of 1.04% at an FPR of 10^{-2} against DeepCorr, but it still performs comparably to random guessing. Remarkably, the latency overhead in this scenario is marginal, showing approximately $0.5ms$ for mean latency and $0.8ms$ for P90 latency. This evaluation clearly demonstrates that MUFLER can dynamically adjust the number of virtual connections according to network conditions to balance security needs with performance, making it a viable solution for enhanced Tor traffic obfuscation. Note that we exclude a similar evaluation for the splitting factor (β), as it shows analogous results.

V. RELATED WORK

Packet-level obfuscation utilizes strategies such as padding and timing delays to standardize packet rates and timings, thereby concealing exploitable patterns for correlation attacks. Studies like BuFLO [6] and CS-BuFLO [42] transmit fixed-length packets at regular intervals to standardize the appearance of traffic, while obfs4 [13] employs encryption and

padding to render Tor traffic indistinguishable from regular internet traffic. Similarly, WTF-PAD [8] adapts padding based on observed outgoing traffic patterns in Tor, inserting dummy messages to obscure statistically unlikely packet delays. FRONT [10] obfuscates the initial parts of traffic traces by adding dummy packets at the middle relay in the Tor network, which are subsequently removed before reaching the destination, ensuring that the packet count and distribution are randomized for each trace. Surakav [43] adopts a machine learning-based approach to obfuscate original traffic patterns by mimicking the patterns of different applications using a GAN-based pattern generator. While these methods enhance privacy, BuFLO and CS-BuFLO can significantly increase bandwidth usage, which is challenging for a network reliant on volunteer resources. Moreover, the need for Tor protocol modifications to remove dummy packets, along with FRONT’s non-inline operation, presents deployment challenges. Surakav offers diverse defense strategies but still incurs high network overhead due to padding and timing delays.

Alternative approaches adopt routing-level obfuscation. For example, TrafficSliver [11] distributes TCP traffic across multiple Tor circuits with unique entry points to prevent flow analysis attacks, but also necessitate Tor protocol modifications, posing deployment hurdles within the existing Tor architecture. Recent methods such as DFD [9] and BLANKET [12] employ adversarial examples to counteract deep learning-based fingerprinting attacks [44], [45] by disrupting recognizable traffic patterns or creating adversarial perturbations. However, these techniques require detailed knowledge of the adversaries’ DNN models and hyperparameters, which limits their practical application and widespread use in real-world scenarios.

VI. CONCLUSION AND FUTURE WORK

We introduce MUFLER, a novel obfuscation system to protect Tor egress traffic from flow correlation attacks. By dynamically shuffling and splitting packets from real connections to a set of virtual connections, MUFLER conceals correlation features effectively without the need for padding data or timing delays. Evaluations show that MUFLER prevents state-of-the-art flow correlation attacks, achieving a TPR of 1% at an FPR of 10^{-2} , while imposing only a 2.4% bandwidth overhead and 27x lower latency overhead than existing solutions.

While MUFLER effectively prevents flow correlation attacks on Tor, its robustness can be improved. For example, when there is a single real connection between the final Tor node and the destination service, the distribution of packets across virtual connections may reveal patterns to adversaries. Future work could address this by fragmenting packets in the real connection into smaller segments, embedding reconstruction metadata in segmented packets, and splitting them as new packets across multiple virtual connections. This approach prevents adversaries from identifying original traffic patterns from virtual connections, even when there is a single real connection. Although this method involves extra headers and metadata for segmented packets, it remains more efficient than existing solutions that rely on hundreds of bytes of padding.

REFERENCES

- [1] M. Seo, M. You, T. Park, S. Shin, and J. Kim, "Poster: Towards a secure and practical system to obfuscate tor network traffic," 2023. [Online]. Available: <https://doi.org/10.5281/zenodo.8197630>
- [2] "Tor Metrics," <https://metrics.torproject.org/>, 2023.
- [3] R. Dingledine, N. Mathewson, P. F. Syverson *et al.*, "Tor: The second-generation onion router," in *USENIX security symposium*, vol. 4, 2004, pp. 303–320.
- [4] S. J. Murdoch and G. Danezis, "Low-cost Traffic Analysis of Tor," in *IEEE Symposium on Security and Privacy*, 2005.
- [5] "Tor Circumvention," 2023, <https://tb-manual.torproject.org/circumvention/>.
- [6] K. P. Dyer *et al.*, "Peek-a-boo, I Still See You: Why Efficient Traffic Analysis Countermeasures Fail," in *IEEE Symposium on Security and Privacy*. IEEE, 2012.
- [7] X. Cai, R. Nithyanand, T. Wang, R. Johnson, and I. Goldberg, "A systematic approach to developing and evaluating website fingerprinting defenses," in *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*, 2014, pp. 227–238.
- [8] M. Juarez, M. Imani, M. Perry, C. Diaz, and M. Wright, "Toward an efficient website fingerprinting defense," in *Computer Security—ESORICS 2016: 21st European Symposium on Research in Computer Security, Heraklion, Greece, September 26-30, 2016, Proceedings, Part I 21*. Springer, 2016, pp. 27–46.
- [9] A. Abusnaina, R. Jang, A. K. Thormali, D. Nyang, and D. Mohaisen, "Dfd: Adversarial learning-based approach to defend against website fingerprinting," in *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*. IEEE, 2020, pp. 2459–2468.
- [10] J. Gong and T. Wang, "Zero-delay lightweight defenses against website fingerprinting," in *29th USENIX Security Symposium (USENIX Security 20)*, 2020, pp. 717–734.
- [11] W. De la Cadena, A. Mitseva, J. Hiller, J. Pennekamp, S. Reuter, J. Filter, T. Engel, K. Wehrle, and A. Panchenko, "Trafficliver: Fighting website fingerprinting attacks with traffic splitting," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020, pp. 1971–1985.
- [12] M. Nasr, A. Bahramali, and A. Houmansadr, "Defeating dnn-based traffic analysis systems in real-time with blind adversarial perturbations," in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 2705–2722.
- [13] "Tor Stats," <https://gitlab.com/yawning/obfs4/blob/master/doc/obfs4-spec.txt>, 2014.
- [14] "ScrambleSuit Protocol Specification," <https://github.com/NullHypothesis/scramblesuit/blob/master/doc/scramblesuit-spec.txt>.
- [15] R. Jansen, T. Vaidya, and M. Sherr, "Point break: A study of bandwidth {Denial-of-Service} attacks against tor," in *28th USENIX security symposium (USENIX Security 19)*, 2019, pp. 1823–1840.
- [16] L. Yang and F. Li, "mtor: A multipath tor routing beyond bandwidth throttling," in *2015 IEEE Conference on Communications and Network Security (CNS)*. IEEE, 2015, pp. 479–487.
- [17] M. Nasr *et al.*, "DeepCorr: Strong Flow Correlation Attacks on Tor Using Deep Learning," in *ACM SIGSAC Conference on Computer and Communications Security*, 2018.
- [18] S. E. Oh, T. Yang, N. Mathews, J. K. Holland, M. S. Rahman, N. Hopper, and M. Wright, "Deepcoffea: Improved flow correlation attacks on tor via metric learning and amplification," in *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2022, pp. 1915–1932.
- [19] Z. Guan, C. Liu, G. Xiong, Z. Li, and G. Gou, "Flowtracker: Improved flow correlation attacks with denoising and contrastive learning," *Computers & Security*, vol. 125, p. 103018, 2023.
- [20] D. Lopes, J.-D. Dong, P. Medeiros, D. Castro, D. Barradas, B. Portela, J. Vinagre, B. Ferreira, N. Christin, and N. Santos, "Flow correlation attacks on tor onion service sessions with sliding subset sum," in *Proceedings of the Network and Distributed System Security (NDSS) Symposium*, 2024.
- [21] "eBPF Introduction, Tutorials," <https://ebpf.io/>.
- [22] Y. Sun *et al.*, "RAPTOR: Routing Attacks on Privacy in Tor," in *24th USENIX Security Symposium*, 2015.
- [23] M. K. Wright *et al.*, "An Analysis of the Degradation of Anonymous Protocols," in *NDSS Symposium*, 2002.
- [24] R. Jansen, M. Juarez, R. Galvez, T. Elahi, and C. Diaz, "Inside Job: Applying Traffic Analysis to Measure Tor from Within," in *NDSS*, 2018.
- [25] N. Borisov, G. Danezis, P. Mittal, and P. Tabriz, "Denial of service or denial of security?" in *Proceedings of the 14th ACM conference on Computer and communications security*, 2007, pp. 92–102.
- [26] N. Feamster and R. Dingledine, "Location diversity in anonymity networks," in *Proceedings of the 2004 ACM workshop on Privacy in the electronic society*, 2004, pp. 66–76.
- [27] S. J. Murdoch and P. Zieliński, "Sampled traffic analysis by internet-exchange-level adversaries," in *International workshop on privacy enhancing technologies*. Springer, 2007, pp. 167–183.
- [28] A. Johnson, C. Wacek, R. Jansen, M. Sherr, and P. Syverson, "Users get routed: Traffic correlation on tor by realistic adversaries," in *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*, 2013, pp. 337–348.
- [29] R. Nithyanand, O. Starov, A. Zair, P. Gill, and M. Schapira, "Measuring and mitigating AS-level adversaries against Tor," in *NDSS Symposium*, 2016.
- [30] "The Atlantic. The creepy, long-standing practice of undersea cable tapping." <https://www.theatlantic.com/international/archive/2013/07/the-creepy-long-standing-practice-of-undersea-cable-tapping/277855/>, 2022.
- [31] "Torrc: Entry nodes," <https://manpages.debian.org/buster/tor/torrc.5.en.html>, 2023.
- [32] "Tor Exit Enclave." <https://help.duckduckgo.com/duckduckgo-help-pages/privacy/tor-exit-enclave/>, 2022.
- [33] S. Kim, J. Han, J. Ha, T. Kim, and D. Han, "Sgx-tor: A secure and practical tor anonymity network with sgx enclaves," *IEEE/ACM Transactions on Networking*, vol. 26, no. 5, pp. 2174–2187, 2018.
- [34] S. Qi, L. Monis, Z. Zeng, I.-c. Wang, and K. Ramakrishnan, "Spright: extracting the server from serverless computing! high-performance ebpf-based event-driven, shared-memory processing," in *Proceedings of the ACM SIGCOMM 2022 Conference*, 2022, pp. 780–794.
- [35] S. Qi, Z. Zeng, L. Monis, and K. Ramakrishnan, "Middlenet: A unified, high-performance nfv and middlebox framework with ebpf and dpdk," *IEEE Transactions on Network and Service Management*, 2023.
- [36] "HAProxy - The Reliable, High Perf. TCP/HTTP Load Balancer," <https://github.com/haproxy/haproxy>, 2024.
- [37] V.-H. Tran and O. Bonaventure, "Making the linux tcp stack more extensible with ebpf," in *Proc. of the Netdev 0x13, Technical Conference on Linux Networking*, 2019.
- [38] W. Yang, P. Chen, G. Yu, H. Zhang, and H. Zhang, "Network shortcut in data plane of service mesh with ebpf," *Journal of Network and Computer Applications*, vol. 222, p. 103805, 2024.
- [39] R. Meier *et al.*, "ditto: WAN Traffic Obfuscation at Line Rate," in *NDSS Symposium*, 2022.
- [40] "Sock Shop: A Microservice Demo Application," <https://github.com/ocp-power-demos/sock-shop-demo>.
- [41] "Robot Shop: Sample Microservice Application," <https://github.com/instana/robot-shop>.
- [42] X. Cai, R. Nithyanand, and R. Johnson, "Cs-bufflo: A congestion sensitive website fingerprinting defense," in *Proceedings of the 13th Workshop on Privacy in the Electronic Society*, 2014, pp. 121–130.
- [43] J. Gong, W. Zhang, C. Zhang, and T. Wang, "Surakav: Generating realistic traces for a strong website fingerprinting defense," in *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2022, pp. 1558–1573.
- [44] M. Seo, J. Kim, E. Marin, M. You, T. Park, S. Lee, S. Shin, and J. Kim, "Heimdallr: Fingerprinting sd-wan control-plane architecture via encrypted control traffic," in *Proceedings of the 38th Annual Computer Security Applications Conference*, 2022, pp. 949–963.
- [45] M. Seo, J. Kim, M. You, S. Shin, and J. Kim, "gshock: A gnn-based fingerprinting system for permissioned blockchain networks over encrypted channels," *IEEE Access*, 2024.